🚩 Git Branching Strategy

Current Strategy

> Currently based on GitFlow, copied from the dev code of contact

Branching Structure

1. **Production Branches (`main` or `master`)**: These branches are used for production code and are typically protected to prevent accidental deletions or unauthorized changes. This is a standard practice in version control to ensure the stability of your production environment.
2. **Development Branch (`development`)**: The development branch serves as the main branch for ongoing development work. While peer review is not always required, it's encouraged to maintain code quality and consistency. Developers have more freedom to work directly in this branch, making it their development playground.
3. **Sprint Branch (`qa`)**: The `qa` branch for the current sprint is a great way to isolate and focus on the work for that specific sprint. It acts as a staging area for sprint-related changes. This branch is used for show-and-tell purposes, and senior team members merge it into the corresponding `sprint-XXXX` branch.
4. **Sprint-Specific Branch (`sprint-XXXX`)**: Each sprint has its own branch, created from the `qa` branch. This approach keeps work organized by sprint and allows for easy tracking and integration of changes specific to that sprint.
5. **Sandbox Environment:** Creating a sandbox environment for training and demos from the `main` or `master` branch with demo data is a practical approach. It ensures that you are working with production code and data for training and demonstration purposes.
6. **Repository Naming Convention:** It's noted that some older repositories use `master` instead of `main` for the main branch. This is in line with historical practices, and many repositories are transitioning to use `main` as the default branch name, which aligns with current best practices.

⚠️ This document outlines our development workflow, from branch creation to code commits and reviews. Following these practices helps maintain code integrity and facilitates collaboration within the team.

Branching and Merge Strategy

This document outlines our team's workflow for managing branches and code integration in our Git repository. This strategy is designed to ensure code quality, collaboration, and a smooth development process.

Branches

1. **Production Branch**: `main` or `master`
   - Represents the production-ready codebase.
2. **Development Branch**: `development`
   - Used for ongoing development and integration of features and bug fixes.
3. **QA Branch**: `qa`
   - Reserved for testing purposes and staging before code promotion to the master branch.

Creating New Branches

- New branches must be created from the **master** branch to ensure a stable starting point.

Merging Strategy

- To merge code into various branches (e.g., development, master, QA), always use a **merge request** (pull request) strategy. This ensures formal code review and integration.

Creating Branches from QA

- When new code is required for development that isn't yet in the master branch, create feature branches from the **QA branch**.

A more detailed explanation of when and why feature branches are created from the QA branch. Click on *"see details"* below.

<details>
<summary>see details</summary>

1. **Feature Development:** Feature branches are created from the QA branch when you are working on a new feature or enhancement that is part of the project's roadmap but is not yet ready for production. By creating a feature branch, you isolate the development of this feature, allowing you to work on it independently from other ongoing development efforts. This separation helps prevent the introduction of incomplete or unstable code into the main development branch, which could be `main` or `master` branches depending on the repository main protected branch name given, ensuring code stability.
2. **Bug Fixes within a QA Cycle:** During a QA cycle, the QA team may identify defects or issues in features that are already in the QA branch. In such cases, it's beneficial to create feature branches specifically for fixing these identified bugs. This approach allows developers to focus on resolving the issues without interfering with the ongoing development work in the QA branch. Once the bug fixes are complete, they can be merged back into the QA branch for further testing.
3. **Hotfixes:** Hotfix branches are created directly from the master branch when critical production issues arise that require immediate attention. Hotfixes are typically unrelated to ongoing QA branch work. By creating hotfix branches from the master branch, you ensure that the fixes are based on the latest stable production code, minimizing the risk of introducing unintended changes from the QA branch.
4. **Experimentation and Prototyping:** Feature branches can also be useful for experimentation and prototyping. If you have ideas for new features or improvements but are uncertain about their impact, you can create a feature branch to experiment with these ideas. This allows you to assess feasibility and gather feedback without affecting the main development or production branches.

</details>

5. **Release Candidate Testing:** When preparing for a release, you may create a release candidate branch from the QA branch. This branch includes the latest features and bug fixes intended for the upcoming release. It serves as a staging environment for final testing and quality assurance before the release is deployed to production.
6. **Parallel Development:** In situations where multiple teams or developers are working on different features concurrently, feature branches are essential for parallel development. Each team can create its feature branch from the QA branch to develop and test their respective features independently. This approach prevents conflicts and facilitates collaboration among teams.
7. **Hotfixes for Production:** ***For critical production issues that require immediate resolution and are unrelated to ongoing QA branch work, hotfix branches should be created directly from the master branch. This ensures that hotfixes are based on the latest stable production code and can be deployed quickly to address urgent issues without waiting for ongoing QA cycle code to be merged into the master branch.***

These practices help maintain code quality, stability, and organization within a version control workflow, ensuring that different types of work, including feature development, bug fixes, hotfixes, and experimentation, can be managed effectively without causing conflicts or compromising production stability.

No Direct Local Merging

- Avoid direct local merges into development, master, or QA branches.

Daily Push to Development

- At the end of each day, push the current state of your feature or ticket branch to the remote repository to keep development up-to-date.

Merge Request Approval

- Code changes that require merging must go through a **merge request** process.
- At least **one approval** from a peer is required after code review before merging.

Preventing Pull from Development into Ticket Branches

- Development code should never be directly pulled into any ticket or feature branches.

Scenarios

Scenario 1: Working on Ticket SENS-1234

- Create a branch (`SENS-1234`) from `master` .
- Develop the code for the ticket and push to the remote branch.

- Create a merge request from `SENS-1234` to `development`. Peer review is recommended but not required.

Scenario 2: Working on Ticket SENS-other, Need Code from SENS-1234

- Create a branch (`SENS-other`) from `SENS-1234`.
- Develop the code for the new ticket and push changes.
- Create a merge request from `SENS-other` to `development`. Peer review is recommended but not required.

Scenario 3: Code of SENS-1234 is Ready for QA

- Create a merge request from `SENS-1234` to `qa`.
- Two developers must perform a peer review, with one approval required. The code cannot be reviewed and approved by the same developer.

Scenario 4: Starting a New Ticket (e.g., SENS-111)

- Create a branch (`SENS-111`) from `master` as no new code has been merged into `qa` for the current sprint.
- Develop the code for the new ticket, pushing changes daily.
- Create merge requests from `SENS-111` to `development` and `SENS-111` to `qa`. Peer review is recommended but not required for both merge requests.

Final Sprint Code Merge

- When the sprint code is complete and tested in the `sprint-XXXX` branch, create merge requests from `qa` to the `sprint-XXXX` branch and from the `sprint-XXXX` branch to merge the final sprint code into the production `master` branch.
- Two developers must perform a final review and approval, and only senior developers, the project manager, or designated personnel can execute the merge request.

This document serves as a reference for our Git workflow and branching strategy. It helps maintain code quality, collaboration, and efficient code integration.

General Workflow

1. **Pull Latest Changes from QA**:
   - Before starting your work, ensure you have the latest changes by pulling from the `qa` branch.
2. **Jira Ticket Assignment**:

- Assign a Jira ticket to yourself and mark it as "In Progress" to indicate that you're actively working on it.

3. **Create Feature Branch**:
   - Create a new local branch named `feature/sens-xxxx-feature-name` based on the Jira ticket you're working on. For example, if the Jira ticket is `SENS-54` titled "Add feature Foo with Bar capability," your branch name should be `feature/sens-54-foo-with-bar`.
4. **Regular Commits and Pushes**:
   - Commit and push your work regularly, at least once a day. Avoid leaving work uncommitted at the end of your workday, and ensure you don't leave commits locally without pushing them to the remote repository.
   - Each commit message must contain:
     - The Jira ticket number you're working on (e.g., `SENS-54`).
     - A concise description of what you did (e.g., "SENS-54 #time 45m Started work on Foo feature and added the landing page").
   - If your work relates to a subtask, include both the subtask and parent task in the commit message (e.g., "SENS-54 SENS-60 #time 6h Added Bar capability to the Foo feature").
   - Keep each commit focused on as few Jira tickets as possible. Avoid monolithic commits that complete multiple Jira tickets.
5. **Feature Review (if needed)**:
   - If your feature requires review, create a new pull request (PR) from your feature branch into the `development` branch. Mark your Jira ticket as needing review.
   - The PR should include a thorough explanation of your changes, what needs to be tested, and step-by-step instructions for another engineer to review and test your work.
6. **Bug Fixes and Issue Resolution**:
   - In cases where a critical bug is fixed or the issue is not self-evident, add a comment to the Jira ticket. Explain what went wrong, and provide details about the steps you took to resolve the issue.

Planned Strategy A

> Based around Trunk-based Development

Prerequisites

- Feature flags

Main changes

-

Planned Strategy B

> Based around GitLab Flow

Prerequisites

-

Main changes

-